

Phase 4 Functional Validation and Comparison Report

1. Phase 4 Objective

The assignment requires the original and refactored versions to be executed and compared to demonstrate that refactoring did not change original functionality. The required evidence is:

- Execution screenshots or logs.
- Test results, when available.
- A comparison report showing unchanged behavior.

This folder provides all three forms of evidence through reproducible execution logs, JUnit results, and this comparison report.

2. Validated Change

The validated production file is:

```
jgrapht-core/src/main/java/org/jgrapht/alg/flow/BoykovKolmogorovMFIImpl.java
```

Original revision:

```
3215ef7eebdbaaf11f95d92d614fc552e8d9006e
```

Refactored revision:

```
6183e894718be159d82a2182a72ca3bfb08b2a90
```

The committed refactor changes the private `adopt()` implementation. It extracts:

- Orphan selection into `nextOrphan()`.
- Source-tree handling into `adoptSourceOrphan()`.
- Sink-tree handling into `adoptSinkOrphan()`.
- Direction-specific parent searches.
- Direction-specific free-vertex processing.
- Common free and adoption assignments.

The earlier Phase 3 proposal also discussed refactoring `grow()`, but the committed revision does not change `grow()`. Phase 4 therefore validates the actual committed `adopt()` refactor only.

3. Functional Invariants

The refactor was expected to preserve:

- Public API signatures.
- Maximum-flow values.
- Source-tree and sink-tree edge-orientation rules.
- Orphan and child-orphan processing order.
- Parent selection by minimum distance.
- Timestamp and distance updates.
- Neighbor activation and child-orphan creation.
- Directed and undirected graph behavior.

The Git diff contains one changed production file and no test or public-interface changes. The changed methods and the new AdoptionParent helper are private implementation details.

4. Validation Method

4.1 Original Version

The original revision was checked out in a separate detached Git worktree:

```
C:\tmp\jgrapht-phase4-master
```

Command:

```
mvn -pl jgrapht-core `
  "-Dtest=org.jgrapht.alg.flow.BoykovKolmogorovMFImpTest" `
  test
```

4.2 Refactored Version

The same focused command was run on the refactored branch:

```
mvn -pl jgrapht-core `
  "-Dtest=org.jgrapht.alg.flow.BoykovKolmogorovMFImpTest" `
  test
```

4.3 Broader Regression Validation

The refactored revision was also tested against all selected maximum-flow implementation tests:

```
mvn -pl jgrapht-core `
  "-Dtest=org.jgrapht.alg.flow.*MFImpTest,org.jgrapht.alg.flow.MaximumFlowAlgorithmTest" `
  test
```

5. Before and After Results

Validation	Original version	Refactored version	Comparison
Focused tests run	31	31	Identical
Focused failures	0	0	Identical
Focused errors	0	0	Identical
Focused skipped	0	0	Identical
Focused build result	Success	Success	Identical
Broader maximum-flow tests	Not required for baseline comparison	102 passed	No regression detected
Public API changed	No	No	Identical

The different elapsed times are not behavioral differences. The original run used a clean worktree and recompiled all core and test sources, while the refactored worktree already had compiled artifacts.

6. Representative Output Comparison

`BoykovKolmogorovMFImpTest.test1()` constructs the same directed and undirected weighted graph in both revisions and asserts:

```
Directed maximum-flow value: 19
Undirected maximum-flow value: 34
```

Because `test1()` passed in both 31-test runs, both versions produced those same expected values. The remaining tests similarly passed their expected maximum-flow assertions in both versions.

7. Behavior-Preservation Analysis

The extracted source-tree parent search preserves the original conditions:

- The inverse edge must have residual capacity.
- The candidate must be a source-tree vertex.
- The candidate must remain connected to the terminal.
- The candidate with the smallest distance is selected.
- The inverse edge is stored as the parent edge.

The extracted sink-tree parent search preserves the symmetric rules:

- The forward edge must have residual capacity.
- The candidate must be a sink-tree vertex.
- The candidate must remain connected to the terminal.
- The candidate with the smallest distance is selected.
- The forward edge is stored as the parent edge.

The refactor also preserves child-orphan priority, list/dequeue operations, active-neighbor checks, parent-edge identity checks, timestamp changes, and distance assignment. The passing directed and undirected tests provide executable evidence that these directional rules remain correct.

8. Evidence Files

The Phase 4 evidence is organized as follows:

```
project_submission/Phase4/  
  functional_validation_comparison_report.md  
  phase4_functional_validation_report.pdf  
  original_version_execution_log.txt  
  refactored_version_execution_log.txt  
  maximum_flow_regression_test_log.txt
```

- `original_version_execution_log.txt` records the fresh baseline run.
- `refactored_version_execution_log.txt` records the matching post-refactor run.
- `maximum_flow_regression_test_log.txt` records the broader 102-test regression run.

9. Conclusion

The original and refactored versions both compile and pass the same 31 focused BoykovKolmogorovMFMPLTest tests with zero failures, zero errors, and zero skipped tests. The refactored revision additionally passes 102 maximum-flow regression tests.

Therefore, the available functional evidence demonstrates that the committed orphan-adoption refactor preserves the original maximum-flow outputs and externally observable behavior.